



Research Project (PRe)

Specialization: Mathematics & Computer Science

Academic Year: 2024–2025

Dynamic Forward Pass Scheduling for Instance-Dependent Early Stopping

Confidentiality Statement

This document is non-confidential.

Author: Felix Azian

Class: 2026

ENSTA Paris Supervisor: Mr. Zacharie Ales

Host Organization Supervisor: Mr. Suqin Yuan

Internship conducted from 26/05/2025 **to** 23/08/2025

Name of Host Organization: University of Sydney – AI Centre

Address: J12/1 Cleveland St, Darlington NSW 2008, Australia

Confidentiality Note: This internship report is non-confidential

Résumé

Ce projet de recherche étend l'Instance-dependent Early Stopping (IES) en intégrant une planification dynamique des passes avant afin de réduire le coût computationnel dans les tâches de classification tout en maintenant les performances. IES utilise la différence seconde de la fonction de perte pour refléter la dynamique d'apprentissage propre à chaque instance et pour exclure de la rétropropagation les instances maîtrisées. Cependant, il faut encore effectuer une passe avant à chaque époque pour suivre la fonction de perte, ce qui peut représenter jusqu'à 25% du coût total. Dans ce projet, nous adaptons dynamiquement la fréquence des passes avant pour les instances maîtrisées en fonction de leur stabilité. Nous analysons ensuite l'efficacité de passer de la différence du second ordre à celle du premier ordre lors des phases avancées de l'entraînement. Enfin, nous explorons l'application de l'IES au réglage fin supervisé (SFT) des LLM et évaluons son impact sur l'efficacité de l'entraînement.

Mots-clés : Apprentissage profond, Passe avant, Maîtrisé, Dynamique, Critère

Abstract

This research project extends Instance-dependent Early Stopping (IES) by integrating a dynamic forward-pass schedule to save computation in classification tasks while maintaining performance. IES uses the second-order difference of the loss to reflect per-instance learning dynamics and to exclude mastered instances from backpropagation. However, it still needs a forward pass at every epoch to monitor loss, which can account for up to 25% of total cost. In this project, we first dynamically adapt the frequency of forward passes for mastered instances based on their stability. This reduces computational cost without sacrificing accuracy. We then analyze the effectiveness of switching from the second-order difference to the first-order difference during the later stages of training. Finally, we explore the possibility of applying IES to Supervised Fine Tuning (SFT) of LLMs and evaluate its impact on training efficiency.

Keywords: Deep Learning, Forward Pass, Mastered, Dynamic, Criterion

Acknowledgments

I would like to sincerely thank the University of Sydney and especially Professor Tongliang Liu for giving me the opportunity to learn with other PhD students at AI Centre. This opportunity represents a milestone in my life, not only as a student from ENSTA Paris, but also as a first-generation Armenian immigrant who once dreamed of working in such a research environment where brilliant minds come together to tackle meaningful challenges.

I would also like to thank Suqin Yuan, who has always been by my side during these three months and has taught me key lessons about the way a researcher must think to move forward and overcome hindrances in this field.

Finally, I want to thank my referent Professor from ENSTA Paris, Professor Zacharie Ales, who has continuously supported me during this experience.

Contents

1	Dynamic Forward Pass Scheduling	8
1.1	Theoretical analysis of IES selection criterion	8
1.1.1	Deep Neural Networks' Behavior	8
1.1.2	Criterion: Second-Order Difference	8
1.1.3	Smoothing with a Moving Average	8
1.2	Implementation of a Dynamic Forward Pass Period	9
1.2.1	Intuition	9
1.2.2	Establishing a Baseline: Static Forward Pass Period	10
1.2.3	Dynamic Forward Pass Period	14
1.2.4	Analysis of Results	15
1.3	Incorporation of Internal Representation in the Dynamic Forward Pass Schedule	16
1.3.1	Forward Pass Period as a function of Learning Decay	16
1.3.2	Adaptive Ratio	18
1.3.3	Benchmarking	22
1.4	Limitations of a dynamic forward pass schedule for IES	23
2	Extending the Dynamic Forward Pass schedule: Switching Derivative Order	24
3	Adapting Instance-Dependent Early Stopping for Fine-Tuning Large Language Models	29
3.1	Framework	29
3.2	Running SFT	30
3.3	Theoretical Limitations of Adapting IES to Causal Language Models	31
4	Appendix	34
4.1	Appendix A: Smoothing of second order derivative - Moving Average	34
4.2	Appendix B: Backpropagation with Stochastic Gradient Descent	34
4.3	Appendix C: Cross-Entropy Loss	35
4.4	Appendix D: Simplified LR based forward pass schedule used for research	36
4.5	Appendix E: Training Arguments for Supervised Fine Tuning	37

Introduction

While it may seem that reducing computational cost is merely a technical or numerical concern, its implications reach far beyond performance metrics. Training large-scale models consumes a significant amount of energy, and improving efficiency is not only a matter of speed, it's also a way to reduce carbon emissions, extend hardware lifespan, and lower the economic and environmental footprint of AI.

This is the main reason that motivates this research project. We need to maintain performance while reducing computational time. The challenge is to find strategies that skip redundant computations without a decrease in accuracy. Instance-dependent Early Stopping (IES) identifies and removes mastered samples from backpropagation. Building on this, our goal is to design dynamic schedules to determine when to re-evaluate these samples. These re-evaluations are essential, however, as the model may forget previously mastered instances while learning new patterns.

1 Dynamic Forward Pass Scheduling

1.1 Theoretical analysis of IES selection criterion

1.1.1 Deep Neural Networks' Behavior

In Machine Learning, when we try to make a model learn patterns out of samples by consistently updating weights of the layers through backpropagation, we observe that the model doesn't learn from the instances uniformly. Indeed, each instance has its own learning trajectory, meaning that while the model might struggle to learn from some samples (e.g. the gradient of the loss function in this direction is high and the loss landscape curvature in this direction is high), it has mastered some easy patterns and instances implying a flat loss landscape curvature.

This is the main idea on which IES relies: The model saves computational time on backpropagation by not considering already mastered instances.

1.1.2 Criterion: Second-Order Difference

To identify the mastered instances, we use the second-order difference, which quantifies the rate of change in the loss for sample i across three consecutive epochs, t^{th} , $(t-1)^{th}$, and $(t-2)^{th}$. The second-order difference is defined as [1]:

$$\begin{aligned}\Delta^2 L_i(w^{(t)}) &= [L_i(w^{(t)}) - L_i(w^{(t-1)})] - [L_i(w^{(t-1)}) - L_i(w^{(t-2)})] \\ &= L_i(w^{(t)}) - 2L_i(w^{(t-1)}) + L_i(w^{(t-2)}).\end{aligned}\tag{1}$$

The reason why we don't look directly at the loss function to see whether or not an instance is mastered is because even if an instance is perfectly learned by the model, its final loss value might still be relatively high compared to another "easy" instance, which means that we would have been compelled to put a specific threshold on each and every instance to know whether or not it is mastered, which is suboptimal.

Thus, we can understand that a low loss value doesn't necessarily mean the model has mastered that specific instance in the sense of no longer needing significant updates. It might just be an inherently easy instance, or the model might have simply memorized it.[1]

The second-order difference, however, measures the change in the rate of change of the loss, which is directly related to the curvature of the loss landscape and gives us a general view of the model's behavior in a given instance's direction depending on how high the curvature is towards a certain curvature.

The most important thing to remember is that this allows for a unified threshold to be applied across all instances, which makes it more robust than trying to set individual loss thresholds for every instance.

1.1.3 Smoothing with a Moving Average

While the second-order difference provides a reliable signal of how much the loss landscape is still changing around a given sample, it can still exhibit high variance due to noise, especially when computed from just three consecutive epochs. To address this, **we do not rely on the raw**

second-order difference directly. Instead, we apply a moving average (see [Appendix A](#)) over the sequence of second-order differences for each instance. This smoothing technique helps to filter out short-term fluctuations and focuses on the general trend in curvature. By averaging the last k values of the smoothed sequence, we obtain a more stable estimate of whether an instance is consistently exhibiting low curvature. This also prevents premature exclusion of samples due to temporary plateaus.

1.2 Implementation of a Dynamic Forward Pass Period

We implemented the **Instance-dependent Early Stopping (IES)** method on standard image classification tasks using the **CIFAR-10** and **CIFAR-100** datasets [2]. Both datasets consist of 60,000 color images of resolution 32×32 pixels. **CIFAR-10** contains 10 classes with 6,000 images per class, while **CIFAR-100** contains 100 classes and 600 images per class. In both cases, the data is divided into 50,000 training images and 10,000 test images.

The backpropagation was performed using **Stochastic Gradient Descent (SGD)** (see [Appendix B](#)) as the optimization algorithm.

The code used to build our dynamic forward pass schedule can be found in [3].

Three learning rate strategies were used in this project:

- **Fixed learning rate**
- **Linear learning rate**
- **Exponential learning rate**

The neural network trained during the process is **ResNet18** [4].

The main hyperparameters used for training are as follows:

- Batch size: 64
- Initial threshold: $\delta = 1 \times 10^{-3}$
- Number of epochs: 200
- Number of iterations: 5
- Moving average rate: 3
- Loss function: Cross Entropy (see [Appendix C](#))

1.2.1 Intuition

It is necessary to periodically evaluate the *mastered* instances during training, as the model might **forget** previously learned examples while acquiring new patterns. This phenomenon, known as *catastrophic forgetting* [5] is particularly common in scenarios involving non-i.i.d. data or unbalanced datasets. Monitoring these instances ensures that they remain correctly learned over time.

Moreover, as the network encounters and learns from more challenging or hard examples, its internal representations may shift, which can degrade performance on previously mastered data.

Intuitively, during the initial phase of training, the model actively learns from all instances, necessitating a forward pass at each epoch to evaluate which instances can be considered as *mastered*. However, as training progresses and the model begins to capture useful patterns, the loss landscape tends to flatten. In this regime, continuously performing forward passes at every epoch becomes suboptimal in terms of computational efficiency.

To address this, we aim to **dynamically reduce the frequency** of forward passes as the training advances. The key challenge lies in striking a balance between maintaining model accuracy and minimizing the overall computational cost.

1.2.2 Establishing a Baseline: Static Forward Pass Period

We first implemented Instance Dependent Early Stopping by looking at the following static periods of forward pass on the excluded samples:

$$[1 , 2 , 3 , 4 , 5 , 10]$$

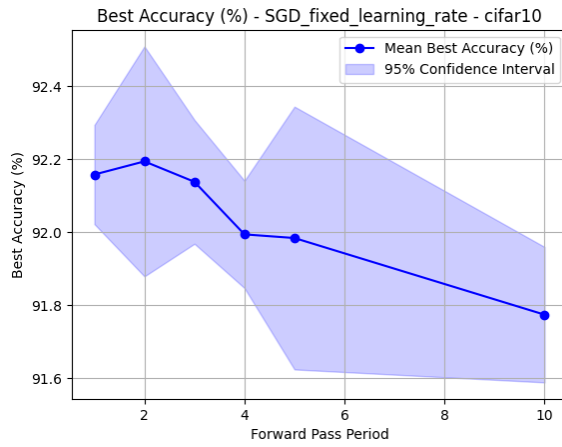
Evaluation Metrics

To assess the effectiveness of the different forward pass schedules, we used three main metrics:

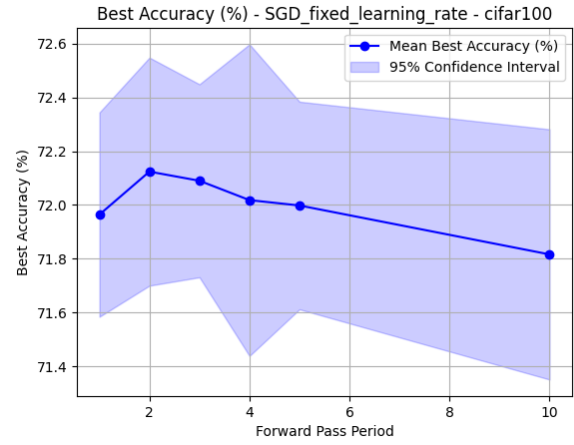
- **Best Test Accuracy (%)**: For each training run, we tracked the classification accuracy on the test set at every epoch and recorded the highest accuracy achieved.
- **Saved Samples Ratio (%)**: At each epoch, we excluded samples identified as “mastered” using a second-order derivative criterion on their loss trajectory. We computed the ratio of total excluded samples over all epochs to the maximum number of training samples processed.
- **Total Running Time (seconds)**: We measured the time taken to complete the learning process. Plotting this value over time allows us to observe how a given forward pass period strategy affects training duration as the model converges.

These metrics together provide a comprehensive view of the trade-off between performance (accuracy), efficiency (saved computation), and training time.

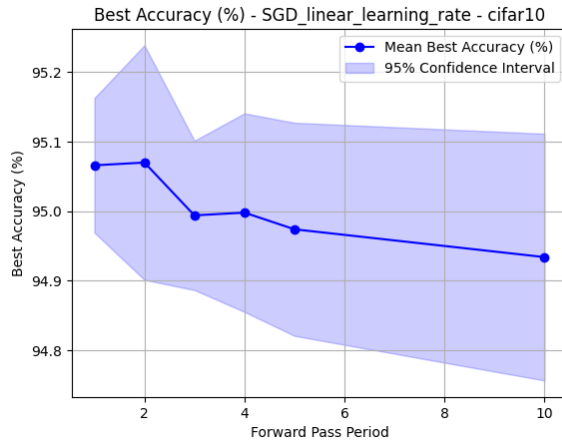
Here is what we observe:



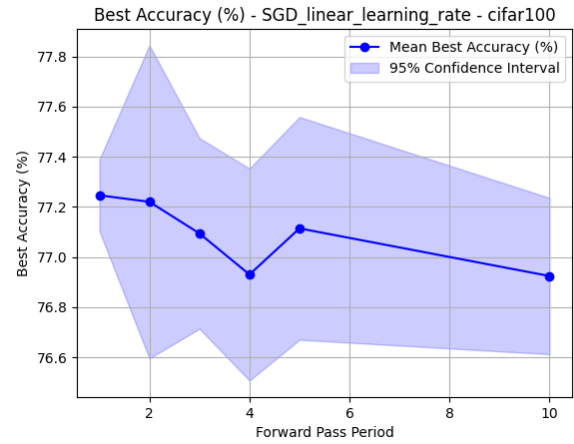
(a) CIFAR10 - Fixed Learning Rate - Accuracy



(b) CIFAR-100 - Fixed Learning Rate - Accuracy



(c) CIFAR-10 - Linear Learning Rate - Accuracy



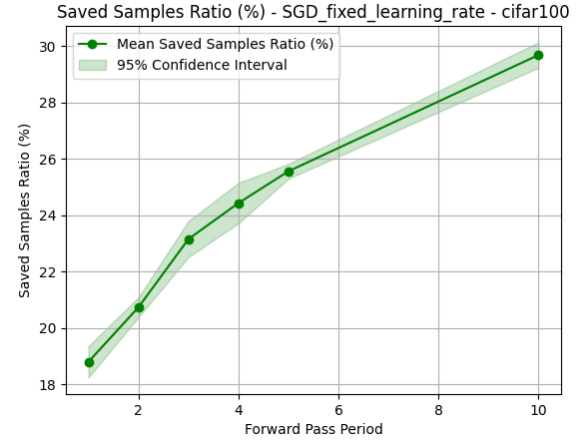
(d) CIFAR-100 - Linear Learning Rate - Accuracy

Figure 1: Comparison of Accuracy across different Static Forward Pass Periods

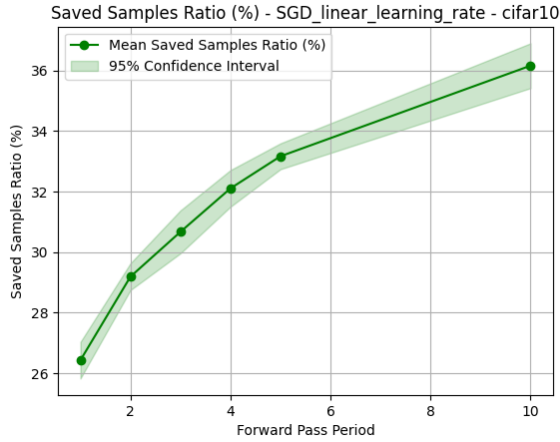
We observe a general downward trend in accuracy as the forward pass period increases, particularly under fixed learning rate conditions. This suggests that longer intervals between forward evaluations can lead to performance degradation. The results under linear learning rate decay show slightly higher overall accuracy, with smaller variations, indicating improved robustness to infrequent updates. Overall, shorter forward pass periods tend to preserve better test performance, especially in more complex datasets like CIFAR-100 which is what we expected.



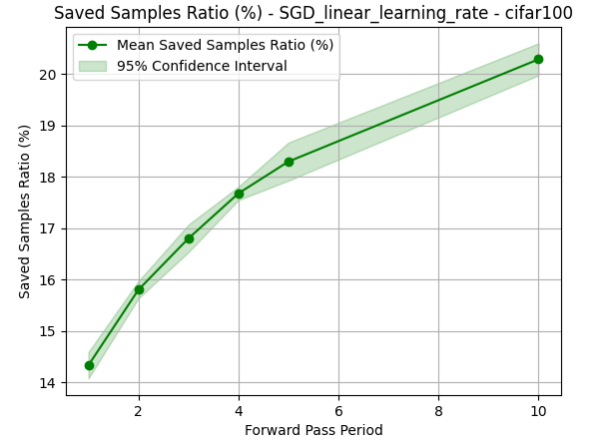
(i) CIFAR-10 - Fixed Learning Rate - Saved Sample Ratio



(j) CIFAR100 - Fixed Learning Rate - Saved Sample Ratio



(k) CIFAR-10 - Linear Learning Rate - Saved Sample Ratio



(l) CIFAR-100 - Linear Learning Rate - Saved Sample Ratio

Figure 2: Comparison of Saved Sample Ratios across different Static Forward Pass Periods

We observe a clear upward trend in all settings: longer forward pass intervals lead to a higher proportion of samples being saved. We can explain this by saying that as we perform forward pass on excluded (mastered) samples only every k epochs, the window of being excluded without re-checking whether or not a given sample is still mastered is longer.

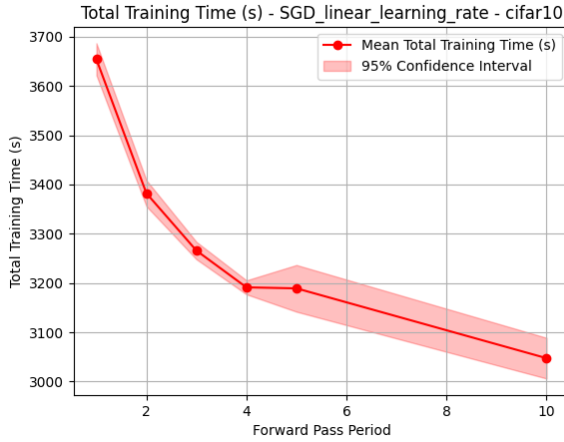
Thus there is an accumulation of excluded instances over time as we create an imbalance, given that samples are still excluded frequently, but they are not reconsidered frequently as we increase the forward pass period on excluded samples.



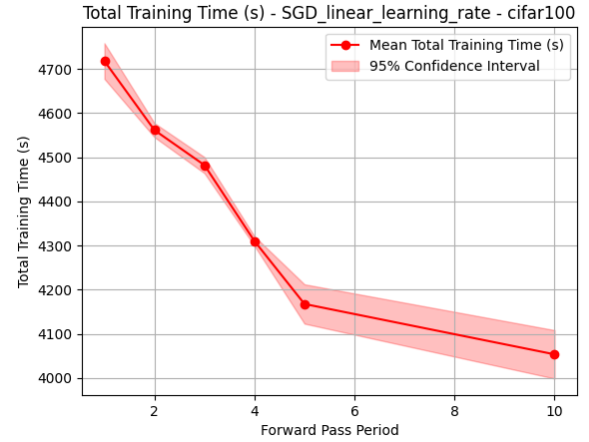
(i) CIFAR-10 - Fixed Learning Rate - Training Time



(j) CIFAR100 - Fixed Learning Rate - Training Time



(k) CIFAR-10 - Linear Learning Rate - Training Time



(l) CIFAR-100 - Linear Learning Rate - Training Time

Figure 3: Comparison of Training Times across different Static Forward Pass Periods

We observe a clear downward trend in training time as the forward pass period increases. This is consistent across all configurations. When k increases, the model performs forward passes on the excluded samples less frequently, thus reducing the computational cost associated with evaluating these instances. For example, with a fixed learning rate on CIFAR-10, increasing k from 1 to 10 leads to a notable drop in training time from approximately 3000s to 2000s. A similar pattern is evident for CIFAR-100. The same trend is also observed with linear learning rates, though overall training times are higher compared to fixed learning rates due to larger learning rate values at the beginning of training. These results confirm that increasing the forward pass period is an effective strategy to significantly reduce training time.

Conclusion

These three metrics showed us the necessity to find a perfect balance between accuracy and computational cost, which leads to the implementation of a dynamic forward pass period.

1.2.3 Dynamic Forward Pass Period

To improve computational efficiency while still monitoring the evolution of "mastered" samples during training, we explore dynamic strategies for setting the period of forward passes (k_{it}) instead of using a constant value. The underlying intuition is that as training progresses and the model stabilizes, checking on previously excluded samples can be done less frequently.

Importance of Plateau Length in Dynamic Scheduling

When designing dynamic update strategies for the forward pass period k_{it} , it is essential to ensure that the function produces sufficiently long plateaus. A plateau is defined as a range of consecutive epochs where k_{it} remains constant.

If the value of k_{it} increases too rapidly (e.g., with $\log(e)$ or $\sqrt{e}$ without scaling), the model may update too frequently during early training, incurring computational cost close to the constant schedule. This defeats the goal of reducing redundant computation.

To address this, we must design or scale dynamic functions so that:

$$k_{it} = \min(k_{\max}, \max(1, \lfloor f(e) \rfloor))$$

where e denotes the current epoch, and $k_{\max}$ is the maximum cap on the forward pass period (e.g., 5). $f(e)$ is chosen such that the increments in k_{it} occur roughly every 10 epochs or more.

Choosing appropriate scalings ensures that forward passes are infrequent enough to save compute but frequent enough to exclude mastered samples efficiently. We propose to benchmark the following functional forms for dynamic periods of forward passes on the excluded samples:

- **Constant:** $k_{it} = 1$
- **Logarithmic:** $k_{it} = \min(k_{\max}, \max(1, \lfloor 2.5 \log(e + 1) \rfloor))$
- **Square-root:** $k_{it} = \min(k_{\max}, \max(1, \lfloor 0.6\sqrt{e + 1} \rfloor))$
- **Linear:** $k_{it} = \min(k_{\max}, \max(1, \lfloor 0.05e \rfloor))$

These functions are compared based on their ability to reduce training time while maintaining performance and effectively filtering stable samples.

To assess the effectiveness of our proposed forward pass schedules for the set of mastered instances, we conducted experiments and compared the results in the following tables:

Table 1: CIFAR-10 — Fixed Learning Rate

FP Strategy	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	92.2 ± 0.1	52.3 ± 0.1	2959.3 ± 32.5
Linear	92.1 ± 0.2	59.3 ± 0.4	2246.6 ± 31.4
Logarithmic	91.9 ± 0.2	60.4 ± 0.3	2166.1 ± 21.2
Square-root	92.0 ± 0.1	60.6 ± 0.3	2152.7 ± 35.7

Table 2: CIFAR-100 — Fixed Learning Rate

FP Strategy	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	72.0 ± 0.2	18.8 ± 0.4	3824.6 ± 57.8
Linear	71.9 ± 0.1	24.9 ± 0.3	3432.8 ± 47.9
Logarithmic	72.3 ± 0.2	18.6 ± 0.1	3764.8 ± 43.7
Square-root	72.0 ± 0.1	26.2 ± 0.6	3370.4 ± 57.7

Table 3: CIFAR-10 — Linear Learning Rate

FP Strategy	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	95.1 ± 0.1	26.43 ± 0.4	3836.2 ± 48.4
Linear	94.9 ± 0.1	32.5 ± 0.2	3441.4 ± 97.1
Logarithmic	95.0 ± 0.1	32.8 ± 0.2	3131.2 ± 40.1
Square-root	95.0 ± 0.1	32.82 ± 0.2	3230.5 ± 35.1

Table 4: CIFAR-100 — Linear Learning Rate

FP Strategy	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	77.3 ± 0.1	14.3 ± 0.2	4490.5 ± 101.0
Linear	77.2 ± 0.3	18.2 ± 0.2	3709.8 ± 45.7
Logarithmic	77.3 ± 0.3	13.1 ± 0.1	4227.4 ± 81.7
Square-root	77.0 ± 0.3	18.3 ± 0.1	3983.5 ± 37.3

1.2.4 Analysis of Results

We observe that dynamic forward pass scheduling enables us to significantly reduce computational cost while maintaining model performance. For instance, on CIFAR-10 with a fixed learning rate, both the logarithmic and square-root schedules achieve a reduction of approximately 800 seconds in training time compared to the constant schedule with a slight decrease in accuracy.

Additionally, the save in computational time is less significant on CIFAR-100. This is likely due to the increased complexity of CIFAR-100, which contains a larger number of classes and requires finer-grained distinctions. As a result, the model takes longer to confidently exclude samples.

The square-root schedule demonstrates consistent performance across both datasets, offering substantial time savings even in the presence of greater class diversity. The linear schedule also yields significant results and especially outperforms other schedules for CIFAR-100 with a linear learning rate.

In summary, dynamically adjusting the forward pass schedule proves effective in reducing training time while preserving accuracy. Among the tested strategies, the square-root schedule offers the most balanced trade-off.

We therefore choose to retain the square-root schedule for benchmarking, due to its ability to consistently reduce computational costs without compromising performance under various settings.

However, a major issue of the logarithmic, linear, and square root schedules is that even though they are based on the idea that as the number of epochs increases, forward passes on excluded samples don't need to be made as frequently as in the early stages of training, they remain independent of the internal representation of the neural network being trained.

1.3 Incorporation of Internal Representation in the Dynamic Forward Pass Schedule

In this subsection, we propose two strategies for defining the forward pass schedule based on how the model stabilizes over the course of training.

We then benchmark these two approaches and provide a general conclusion on which schedule offers the best trade-off in terms of performance and computational cost savings.

1.3.1 Forward Pass Period as a function of Learning Decay

In the previous section, we implemented a forward pass schedule for mastered instances based on the idea that the model becomes increasingly stable over time, making frequent reevaluation of excluded samples unnecessary.

To achieve this, we defined the forward pass period as a slowly increasing function of the current epoch.

However, to make the forward pass period truly dynamic and reflective of the model's learning state, it should instead be defined as a function of the learning rate.

Indeed, when the learning rate is high, weights change rapidly between updates which causes the losses to fluctuate significantly. Therefore, we need to evaluate each instance more frequently, implying a small forward pass period k_{it} .

Conversely, when the learning rate is low, weight updates are smaller and the loss becomes more stable over time, which makes it relevant to increase the interval between forward passes.

Given the settings for the learning rates for SGD (see [Appendix B](#)), and the idea that we need a scaled function of the learning rate in order to have long enough plateaus to allow smooth updates of the forward pass schedule, we propose the following function :

```
def k_it_evolve(lr, optimizer_name="SGD_linear_learning_rate", max_k_it=5, min_k_it=1):
    lr_min = 0.001
    lr_max = 0.1

    # Normalize LR to [0, 1]
    normalized = (lr - lr_min) / (lr_max - lr_min)
    normalized = min(1.0, max(0.0, normalized)) # clamp to avoid overflow

    # Invert so low lr → higher k_it
    normalized = 1 - normalized

    # Adjust shape depending on optimizer type
    if optimizer_name == "SGD_linear_learning_rate":
        # Keep as-is for linear
        adjusted = normalized
    elif optimizer_name == "SGD_exponential_learning_rate":
        # Emphasize fast decay: e.g., square it
        adjusted = normalized ** 2
    else:
        raise ValueError(f"Unsupported optimizer: {optimizer_name}")

    # Map to integer k_it
    k_it = round(adjusted * (max_k_it - min_k_it)) + min_k_it
    return k_it
```

Figure 4: Implementation of LR based forward pass schedule

This approach is robust in the sense that, if we decide to change the number of training epochs, we can easily adjust the value of `max_k_it` to control how widely the forward pass periods are spaced throughout training.

To achieve the goal of making the forward pass period increase depending on how the learning rate decreases, we normalize the learning rate to a $[0, 1]$ interval and reverse the scale so that smaller learning rates correspond to longer forward pass periods.

Given that we cap k_{it} at 5, we design for the research a continuous mapping that smoothly increases the period as the learning rate decays (see [Appendix D](#) for the simplified algorithm used for this research project). This allows us to have better control over forward pass period updates. Saving computational time through dynamic scheduling is only effective when the learning rate is not fixed in this configuration.

The experiments will therefore be conducted with a linear learning rate and an exponential learning rate.

The next subsection proposes another way to incorporate the internal representation of the model within the definition of the forward pass period.

1.3.2 Adaptive Ratio

We now propose a dynamic strategy based on the observed change in the proportion of mastered (excluded) samples.

Every 10 epochs, we compute the **adaptive ratio**:

$$r_{\text{saved}}^{(t)} = \frac{N_{\text{saved}}^{(t)}}{T_{\text{samples}}}$$

where $N_{\text{saved}}^{(t)}$ is the number of newly excluded samples over the 10-epoch block t , and T_{samples} is the total number of training samples in the dataset (in our case, we have 50,000 training images for both CIFAR-10 and CIFAR-100). We define:

$$\Delta r^{(t)} = \left| r_{\text{saved}}^{(t)} - r_{\text{saved}}^{(t-1)} \right|$$

If the variation $\Delta r^{(t)}$ is below a threshold θ , it indicates that the number of newly excluded samples has plateaued. In this case, we increase the forward pass interval k_{it} , allowing the model to run more training steps without re-evaluating mastered samples. Conversely, if $\Delta r^{(t)} \geq \theta$, the model is still actively excluding new samples, so we reduce k_{it} to refresh evaluation more frequently.

We perform this analysis every 10 epochs to introduce plateaus where k_{it} remains constant, preventing instability due to frequent changes. The logic below illustrates how this adaptive update is implemented in our code:

```
# --- Dynamic k_it adjustment logic (every 10 epochs) ---
if args.dynamic_k_mode == "adaptive_ratio" and (epoch + 1) % 10 == 0 :
    # Calculate ratio of samples saved in this block relative to the number of samples
    # that were available for training at the start of this 10-epoch block.
    if remaining_loader is not None:
        num_excluded_end_block = len(remaining_loader.dataset)
    else:
        num_excluded_end_block = 0

    saved_samples_this_block = num_excluded_end_block - num_excluded_start_block
    saved_ratio_block = saved_samples_this_block / len(full_train_dataset)
    dynamic_saved_ratio.append(saved_ratio_block)

    print(f"[DEBUG] Block {(epoch//10 + 1):2d}: start={num_excluded_start_block}, end={num_excluded_end_block}, new excluded={saved_samples_this_block}")
    print(f"Epoch {epoch + 1}: Saved ratio for last 10 epochs: {saved_samples_this_block}/{len(full_train_dataset)} = {saved_ratio_block:.4f}")
    print(f"Here is the list of dynamic saved ratios every 10 epoch: {dynamic_saved_ratio}")

    if len(dynamic_saved_ratio) > 1:
        delta = abs(dynamic_saved_ratio[-1] - dynamic_saved_ratio[-2])
        if delta < args.threshold_ratio:
            current_k_it = min(max_k_it, current_k_it + 1) # Increase k_it, capped at max_k_it
            print(f"Dynamic Saved ratio variation ({delta:.4f}) is below threshold ({args.threshold_ratio}). Increasing k_it to {current_k_it}")

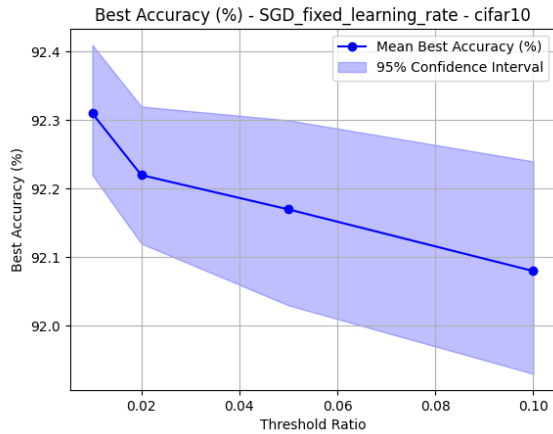
        elif delta >= args.threshold_ratio:
            current_k_it = max(1, current_k_it - 1) # Decrease k_it, min 1
            saved_samples_this_block = 0 #reset for the next epoch
```

Figure 5: Implementation of the Adaptive Ratio method

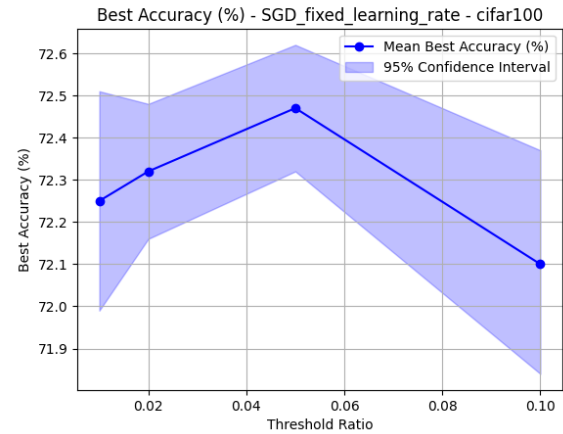
The main concern here is that **implementing this forward pass schedule introduces an additional hyperparameter**: the threshold θ which is used to compare against the absolute variation of the dynamic saved ratio. Therefore, we need to run experiments in order to identify which threshold yields the best Pareto frontier, in order to fix that hyperparameter.

We chose to run our experiments with the following thresholds:

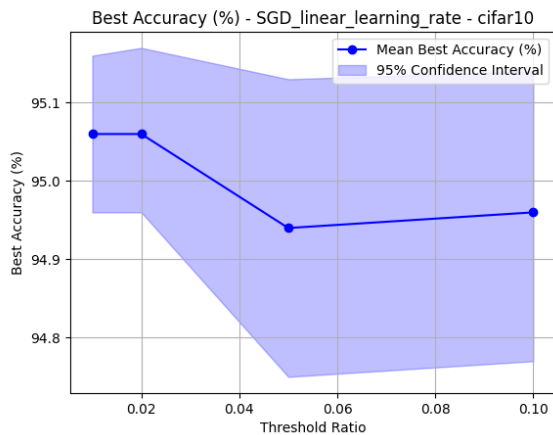
$$[0.01, 0.02, 0.05, 0.1]$$



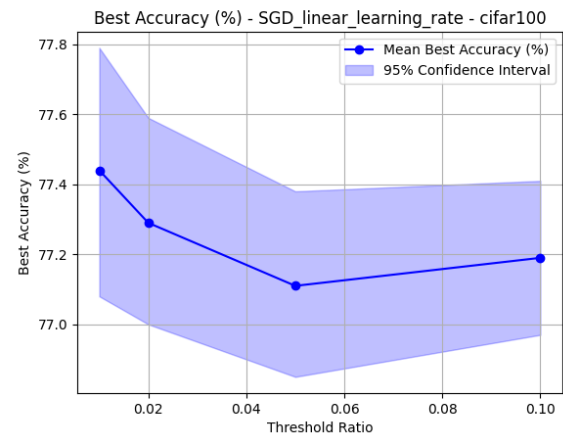
(a) CIFAR10 - Fixed Learning Rate - Accuracy



(b) CIFAR-100 - Fixed Learning Rate - Accuracy

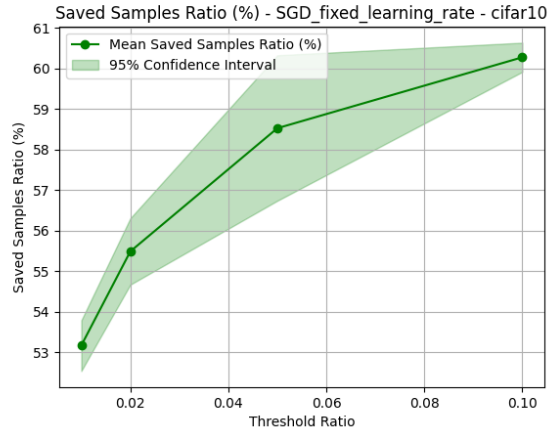


(c) CIFAR-10 - Linear Learning Rate - Accuracy

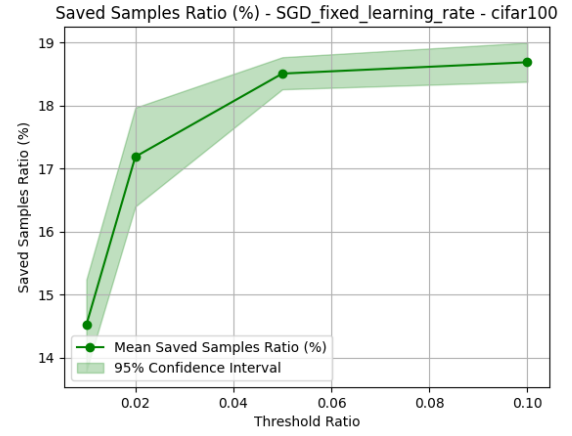


(d) CIFAR-100 - Linear Learning Rate - Accuracy

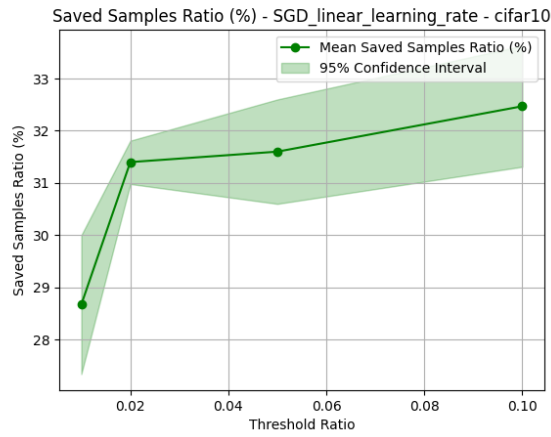
Figure 6: Influence of the threshold's choice for the Adaptive Ratio schedule on Accuracy



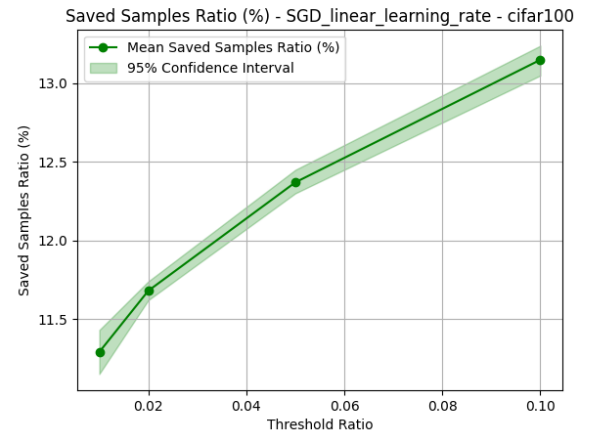
(a) CIFAR10 - Fixed Learning Rate - Saved Sample Ratio



(b) CIFAR-100 - Fixed Learning Rate - Saved Sample Ratio

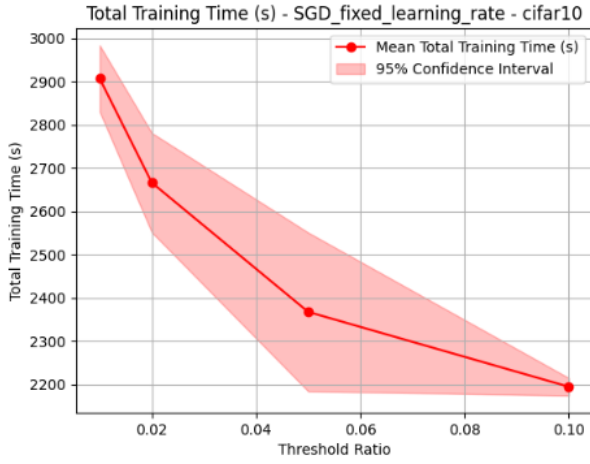


(c) CIFAR-10 - Linear Learning Rate - Saved Sample Ratio

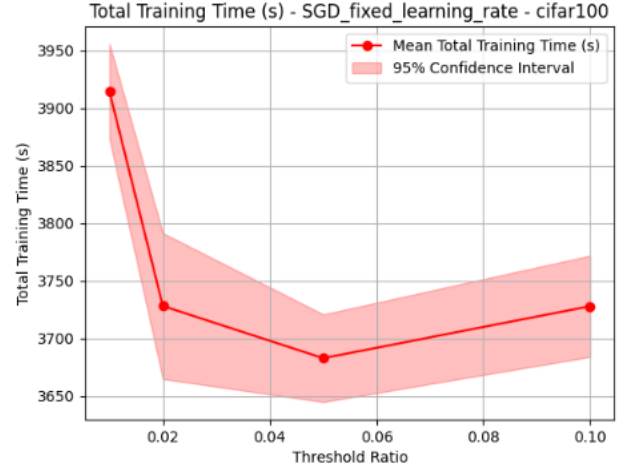


(d) CIFAR-100 - Linear Learning Rate - Saved Sample Ratio

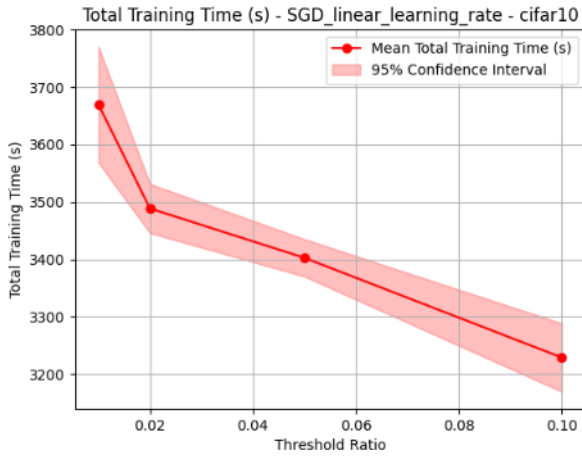
Figure 7: Influence of the threshold's choice for the Adaptive Ratio schedule on Saved Sample Ratio



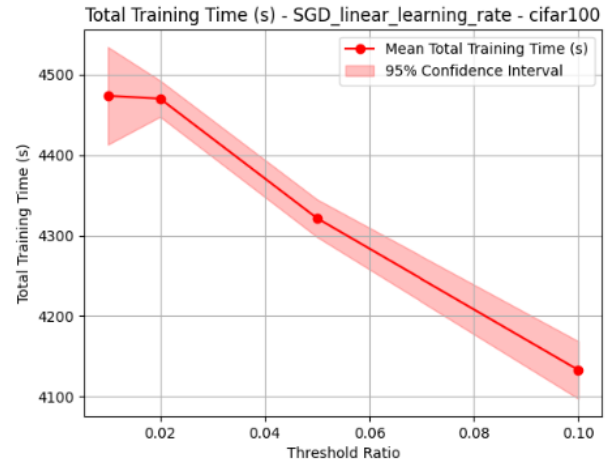
(a) CIFAR10 - Fixed Learning Rate - Total Training Time



(b) CIFAR-100 - Fixed Learning Rate - Total Training Time



(c) CIFAR-10 - Linear Learning Rate - Total Training Time



(d) CIFAR-100 - Linear Learning Rate - Total Training Time

Figure 8: Influence of the threshold's choice for the Adaptive Ratio schedule on Training Time

Based on the results shown in Figures 5, 6, and 7, the threshold ratio of **0.02** appears to be the most balanced choice to save computational time and keep the same level of performance, even though we see that computational time stays particularly high when we use CIFAR-100 and a linear learning rate. Overall, it achieves a significant reduction in total training time for three out of four configurations while maintaining a level of accuracy comparable to that obtained with a constant forward pass period. Additionally, it preserves a reasonable saved sample ratio across both CIFAR-10 and CIFAR-100 datasets, under both fixed and linear learning rate schedules. We can now benchmark the different proposed forward pass schedules and make a general conclusion about which one is the most relevant.

1.3.3 Benchmarking

As the forward pass period defined as a function of the learning rate (this schedule will be referred to as LR) can only be relevant with a non-fixed learning rate, we benchmark LR, Adaptive Ratio and Square Root forward pass schedules for a linear learning rate and for an exponential learning rate in the following tables:

Table 5: CIFAR-10 — Linear Learning Rate - Internal Representation

FP Strategy	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	95.1 ± 0.1	26.4 ± 0.4	3836.2 ± 48.4
LR	95.1 ± 0.1	30.3 ± 0.3	3268.1 ± 34.1
Adaptive Ratio	95.1 ± 0.1	31.4 ± 0.4	3488.7 ± 42.8
Square Root	95.0 ± 0.1	32.8 ± 0.2	3230.5 ± 35.1

Table 6: CIFAR-100 — Linear Learning Rate - Internal Representation

FP Strategy	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	77.3 ± 0.1	14.3 ± 0.2	4490.5 ± 101.0
LR	77.3 ± 0.3	12.6 ± 0.2	4047.1 ± 32.7
Adaptive Ratio	77.3 ± 0.3	11.7 ± 0.1	4470.0 ± 22.5
Square Root	77.0 ± 0.3	18.3 ± 0.1	3983.5 ± 37.3

Table 7: CIFAR-10 — Exponential Learning Rate - Internal Representation

FP Strategy	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	94.7 ± 0.1	55.6 ± 0.6	2937.0 ± 24.6
LR	94.7 ± 0.1	61.4 ± 0.7	2136.9 ± 34.0
Adaptive Ratio	94.7 ± 0.1	63.1 ± 0.6	2070.3 ± 14.3
Square Root	94.6 ± 0.1	62.7 ± 0.8	2364.5 ± 39.6

Table 8: CIFAR-100 — Exponential Learning Rate - Internal Representation

FP Strategy	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	76.9 ± 0.2	17.0 ± 0.2	4061.3 ± 88.0
LR	76.6 ± 0.3	22.4 ± 0.4	3859.9 ± 85.2
Adaptive Ratio	76.6 ± 0.4	23.1 ± 0.5	3857.2 ± 102.0
Square Root	76.7 ± 0.3	23.3 ± 0.4	3945.5 ± 20.0

Analysis

The results presented in Tables 5 to 8 show that both the LR-based and Adaptive Ratio methods are highly effective. For instance, on CIFAR-10 with a linear learning rate, the LR-based strategy achieves the same best accuracy as the constant schedule, but with a higher saved sample ratio and significantly reduced training time (3268.1s vs. 3836.2s). The Adaptive Ratio approach similarly maintains high accuracy while reducing training time.

However, we can see that the Adaptive Ratio method does not yield good results in terms of computational cost saving when combined with a linear learning rate, especially compared to the other forward pass schedules. This can be explained by the fact that a linear learning rate smoothly decays the step size over time. As a consequence, the rate at which new samples are mastered naturally slows down in a stable and predictable manner, already creating plateaus in the exclusion dynamics. Since there are no abrupt changes for the Adaptive Ratio strategy to react to, it cannot effectively adjust k_{it} and therefore fails to provide significant computational savings.

Although the square root schedule also yields strong results, we can argue that since the Adaptive Ratio and LR-based methods adapt to the model’s internal state, they may exhibit better generalization across datasets and learning schedules. However, we observe a clear limitation of the Adaptive Ratio schedule when the learning rate of the chosen optimizer decays progressively. For this reason, we choose to retain the LR-based schedule, which adapts directly to any learning rate and has demonstrated significant results.

1.4 Limitations of a dynamic forward pass schedule for IES

In this first part of the project, we developed an efficient and relevant dynamic forward pass schedule that allows us to save computational time in classification tasks. This was achieved by leveraging the idea that mastered samples do not need to undergo a forward pass at every epoch. As the model stabilizes over time, forward passes on excluded samples can be spaced out more widely.

However, a key limitation of dynamic forward pass scheduling lies in the difficulty of accurately re-evaluating samples that have been excluded for multiple epochs. When a mastered sample is re-evaluated after a prolonged exclusion, increasing the forward pass interval implies a larger temporal gap between recorded loss values. This introduces a **scaling issue**: the second-order loss difference may become distorted due to the longer interval between evaluations. As a result, the signal may appear overly sharp or overly smooth, potentially leading to misclassifications regarding whether a sample should remain excluded or be reintroduced.

An extension of this work could involve switching from second-order to first-order loss differences at a certain point in training. This would reduce the evaluation gaps and yield a finer and more accurate representation of the loss landscape.

2 Extending the Dynamic Forward Pass schedule: Switching Derivative Order

We conduct a series of experiments to evaluate whether switching from second-order to first-order loss differences at a specific point during training can better capture the loss landscape and help address the scaling issue in implementing a dynamic forward-pass schedule for mastered instances.

This **specific point** could be defined in different ways, but we propose here to switch when:

$$k_{it} = \max k_{it}$$

Given the satisfying results of both square root and LR schedules, this final experiment aims to benchmark these two approaches and understand how the switch in derivative order influences the performance of the model.

To support this transition, we introduce an **additional hyperparameter** representing the threshold for the first-order derivative.

We choose the following threshold values for the first-order difference:

$$0.001, 0.005, 0.01, 0.05$$

The following tables summarize our experiments on CIFAR-10 and CIFAR-100 using exponential and linear learning rate schedules. Results are reported for the four first-order (FO) thresholds introduced earlier and are compared against the baseline, which corresponds to a constant forward pass schedule with $k=1$. Based on the most effective threshold, we then benchmark the forward-pass scheduling strategies and present a general conclusion.

Table 9: CIFAR-10 — Exponential Learning Rate - Square Root FP Schedule

FO Threshold	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Baseline	94.7 ± 0.1	55.6 ± 0.6	2937.0 ± 24.6
0.001	94.5 ± 0.1	67.7 ± 1.3	2022.2 ± 17.5
0.005	94.2 ± 0.1	74.9 ± 2.8	1713.5 ± 68.7
0.01	94.1 ± 0.2	77.3 ± 3.4	1689.8 ± 104.1
0.05	93.6 ± 0.2	81.8 ± 4.9	1469.6 ± 143.8

Table 10: CIFAR-100 — Exponential Learning Rate - Square Root FP Schedule

FO Threshold	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Baseline	76.9 ± 0.2	17.0 ± 0.2	4061.3 ± 88.0
0.001	76.4 ± 0.3	34.3 ± 0.6	3281.2 ± 54.1
0.005	75.8 ± 0.2	61.0 ± 1.3	2178.7 ± 24.2
0.01	75.4 ± 0.2	66.2 ± 1.9	2086.9 ± 52.5
0.05	74.6 ± 0.5	73.0 ± 3.5	1759.0 ± 96.7

Table 11: CIFAR-10 — Exponential Learning Rate - LR based FP schedule

FO Threshold	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Baseline	94.7 ± 0.1	55.6 ± 0.6	2937.0 ± 24.6
0.001	94.5 ± 0.2	65.8 ± 2.0	2117.4 ± 29.2
0.005	94.2 ± 0.2	73.3 ± 3.9	1800.1 ± 102.7
0.01	94.2 ± 0.3	75.9 ± 4.7	1802.6 ± 144.6
0.05	93.9 ± 0.4	80.4 ± 6.5	1576.7 ± 191.2

Table 12: CIFAR-100 — Exponential Learning Rate - LR based FP schedule

FO Threshold	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Baseline	76.9 ± 0.2	17.0 ± 0.2	4061.3 ± 88.0
0.001	76.4 ± 0.3	32.8 ± 1.9	3360.3 ± 25.6
0.005	75.9 ± 0.4	58.2 ± 5.1	2310.1 ± 147.4
0.01	75.6 ± 0.3	62.9 ± 6.1	2260.8 ± 199.7
0.05	75.0 ± 0.6	69.5 ± 8.1	1936.1 ± 249.2

Table 13: CIFAR-10 — Linear Learning Rate - Square Root FP schedule

FO Threshold	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	95.1 ± 0.1	26.4 ± 0.4	3836.2 ± 48.4
0.001	94.9 ± 0.1	39.6 ± 0.6	3099.0 ± 43.9
0.005	94.6 ± 0.1	51.5 ± 1.8	2500.5 ± 37.8
0.01	94.5 ± 0.1	56.0 ± 2.2	2459.8 ± 62.9
0.05	94.1 ± 0.1	66.5 ± 4.4	1994.1 ± 126.4

Table 14: CIFAR-100 — Linear Learning Rate - Square Root FP schedule

FO Threshold	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Baseline	77.3 ± 0.1	14.3 ± 0.2	4490.5 ± 101.0
0.001	77.1 ± 0.2	18.8 ± 0.2	3868.8 ± 59.1
0.005	76.5 ± 0.2	31.7 ± 0.4	3217.7 ± 23.4
0.01	76.0 ± 0.3	35.8 ± 0.5	3254.5 ± 13.1
0.05	75.3 ± 0.4	44.7 ± 1.2	2776.1 ± 29.4

Table 15: CIFAR-10 — Linear Learning Rate - LR based FP schedule

FO Threshold	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Baseline	95.1 ± 0.1	26.4 ± 0.4	3836.2 ± 48.4
0.001	95.0 ± 0.1	35.7 ± 1.7	3311.0 ± 7.3
0.005	94.8 ± 0.1	45.3 ± 5.0	2852.4 ± 123.8
0.01	94.6 ± 0.1	49.0 ± 6.5	2883.3 ± 188.6
0.05	94.3 ± 0.2	58.2 ± 10.3	2476.3 ± 283.5

Table 16: CIFAR-100 — Linear Learning Rate - LR based FP schedule

FO Threshold	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Baseline	77.3 ± 0.1	14.3 ± 0.2	4490.5 ± 101.0
0.001	77.2 ± 0.2	17.7 ± 0.7	3958.4 ± 41.8
0.005	76.6 ± 0.2	29.5 ± 1.8	3390.8 ± 20.5
0.01	76.4 ± 0.3	33.0 ± 2.4	3387.2 ± 64.6
0.05	75.6 ± 0.3	39.9 ± 4.6	3039.3 ± 109.1

Analysis: These tables illustrate consistent trends when transitioning from second order to first order loss differences in the identification of mastered samples. Firstly, we observe a significant increase in the *Saved Samples Ratio* when using first-order thresholds, particularly as the threshold value increases. However, this gain comes at the cost of greater variance, as shown by the larger standard deviations, indicating a less stable pruning process.

We note a **clear degradation in Accuracy as the threshold increases**. This decline is expected: higher thresholds allow more samples to be excluded. As a result, the model may prematurely stop learning from instances that still contribute meaningful gradient information, leading to suboptimal generalization.

We choose for the final benchmark a first order threshold of 0.001 because we want to reduce the noise as much as we can while being the closest possible to the baseline in terms of accuracy.

Table 17: CIFAR-10 — Exponential Learning Rate - Final benchmark

FP Strategy	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	94.7 ± 0.1	55.6 ± 0.6	2937.0 ± 24.6
LR (w/o switch)	94.6 ± 0.1	61.4 ± 0.7	2136.9 ± 34.0
LR (w/ switch)	94.5 ± 0.2	65.8 ± 2.0	2117.4 ± 29.2
Square Root (w/o switch)	94.6 ± 0.1	62.7 ± 0.8	2364.5 ± 39.6
Square Root (w/ switch)	94.5 ± 0.1	67.7 ± 1.3	2022.2 ± 17.5

Table 18: CIFAR-100 — Exponential Learning Rate - Final benchmark

FP Strategy	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	76.9 ± 0.2	17.0 ± 0.2	4061.3 ± 88.0
LR (w/o switch)	76.6 ± 0.3	22.4 ± 0.4	3859.9 ± 85.2
LR (w/ switch)	76.4 ± 0.3	32.8 ± 1.9	3360.3 ± 25.6
Square Root (w/o switch)	76.7 ± 0.3	23.3 ± 0.4	3945.5 ± 20.0
Square Root (w/ switch)	76.4 ± 0.3	34.3 ± 0.6	3281.2 ± 54.1

Table 19: CIFAR-10 — Linear Learning Rate - Final benchmark

FP Strategy	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	95.1 ± 0.1	26.4 ± 0.4	3836.2 ± 48.4
LR (w/o switch)	95.1 ± 0.1	30.3 ± 0.3	3268.1 ± 34.1
LR (w/ switch)	95.0 ± 0.1	35.7 ± 1.7	3311.0 ± 7.3
Square Root (w/o switch)	95.0 ± 0.1	32.8 ± 0.2	3230.5 ± 35.1
Square Root (w/ switch)	94.1 ± 0.1	66.5 ± 4.4	1994.1 ± 126.4

Table 20: CIFAR-100 — Linear Learning Rate - Final benchmark

FP Strategy	Best Accuracy (%)	Saved Samples Ratio (%)	Total Time (s)
Constant ($k = 1$)	77.3 ± 0.1	14.3 ± 0.2	4490.5 ± 101.0
LR (w/o switch)	77.3 ± 0.3	12.6 ± 0.2	4047.1 ± 32.7
LR (w/ switch)	77.2 ± 0.2	17.7 ± 0.7	3958.4 ± 41.8
Square Root (w/o switch)	77.0 ± 0.3	18.3 ± 0.1	3983.5 ± 37.3
Square Root (w/ switch)	77.1 ± 0.2	18.8 ± 0.2	3868.8 ± 59.1

Analysis

We observe that switching from the second-order to the first-order derivative yields poor results in terms of capacity to save computational time while maintaining performance. Overall, the LR-based method proves to be the most robust strategy, as it adapts inherently to the learning dynamics of the model and doesn't add a hyperparameter. In practice, it generally outperforms all other approaches.

Conclusion

We saw through this section the hardships of implementing a dynamic forward pass schedule on the set of mastered instances as the model learns patterns from the dataset.

As discussed in [1], larger computational savings can be achieved if we tolerate a performance drop of about 1%. With such a margin, simply increasing the forward pass interval from 1 to 5 already provides substantial reductions in computational cost. However, if the goal is to maintain a performance level comparable to the original IES method while still reducing computation, it becomes necessary to take into account the learning dynamics of the model and avoid scheduling forward passes during phases in which the model is stabilizing.

The most robust way to achieve this is to define the forward pass period as a function of the learning rate of the optimizer used for backpropagation. This approach naturally increases the interval between forward passes as the learning rate decreases, reflecting the fact that the model's learning dynamics slow down in later stages of training.

The main challenge of IES is to adapt it for it to work on Causal Learning models and not just classification tasks. The next section is about exploring the possibility of adapting IES to Large Language Models and especially Supervised Fine Tuning on pre-trained LLMs.

3 Adapting Instance-Dependent Early Stopping for Fine-Tuning Large Language Models

As the scale of Large Language Models (LLMs) continues to grow, the computational cost of Supervised Fine-Tuning (SFT) has become a major concern not only in terms of training time, but also due to the significant energy consumption involved, which raises environmental and sustainability issues. In this section, we explore the hardships and possibilities of adapting IES to the supervised fine-tuning of LLMs, with the goal of reducing the computational cost without compromising model performance.

3.1 Framework

The fine-tuning will be conducted using the LLaMA-Factory framework [6], an open-source infrastructure tailored for training LLMs. We fine-tune the following model within this framework:

- meta-llama/Llama-3.2-1B [7]

The **fine-tuning dataset** is :

- BioInstruct [8]: A biomedical instruction-tuning dataset designed to teach models how to perform various tasks in the biomedical and clinical domain.

We conduct our experiments using **4 NVIDIA 4090 GPUs**.

Supervised Fine-Tuning of LLMs typically involves a small number of training epochs.

In such regimes, the loss does not have enough time to evolve smoothly over long intervals, which makes second-order loss differences particularly sensitive to noise. Therefore, **it may be more robust to rely on the absolute value of the loss or its first derivative as a criterion** for determining whether an instance (a tokenized response) is sufficiently mastered to be excluded from backpropagation.

We will implement **full fine-tuning** instead of LoRA to ensure a complete adaptation of the base models to each task. The size of our model (1.2 billion parameters to train) allows us to make this full fine-tuning and have better performance as we update all the parameters.

The chosen evaluation metric is **ROUGE-L, which is an automatic metric based on the Longest Common Subsequence between a generated text and a reference text..**

ROUGE-L requires generating the full output text for every sample in order to compute the longest common subsequence with the reference, which significantly increases evaluation time.

All training hyperparameters are kept consistent with the default settings provided by the framework to ensure reproducibility and fair comparison.

The overall settings are depicted in [Appendix E](#).

3.2 Running SFT

We fine-tune Llama-3.2-1B on the BioInstruct dataset using the LLaMA-Factory framework. This allows us to later compare the initial convergence pattern to the one obtained with instance-dependent early stopping , and observe how performance and training efficiency are impacted.

```
***** eval metrics *****
epoch                        =          5.0
eval_rougeL                  =         0.4896
eval_runtime                  = 0:14:55.20
eval_samples_per_second      =         27.932
eval_steps_per_second         =          0.437

***** train metrics *****
epoch                        =          5.0
total_flos                    =       90591039GF
train_loss                    =          1.1055
train_runtime                  = 1 day, 2:37:48.35
train_samples_per_second      =          1.304
train_steps_per_second         =          0.082
```

Figure 9: SFT - BioInstruct - Llama-3.2-1B

We can observe how much time it takes to run supervised fine-tuning (SFT) even for just 5 epochs and four powerful NVIDIA 4090 GPUs.

Reducing computational cost while maintaining or improving performance is crucial, especially as models grow in size and deployment becomes more widespread, but there is a clear reason why adapting IES to SFT on LLMs is a real challenge in the field of NLP models.

3.3 Theoretical Limitations of Adapting IES to Causal Language Models

IES was originally developed for classification tasks, where it relies on observing the evolution of the loss through zero, first, or in our case, second order differences, to identify samples that have been sufficiently mastered and can be excluded from further training.

However, adapting IES to the SFT of causal language models presents several theoretical and practical challenges.

1. Fewer Training Epochs in SFT. Unlike classification tasks that typically run over many epochs, SFT for LLMs is usually conducted over a limited number of epochs (often fewer than 5). As a result, IES, which relies on gradually detecting mastered samples over time, has limited opportunity to observe meaningful changes in the loss dynamics. This constraint significantly reduces the effectiveness of IES and limits the potential computational savings it was designed to offer.

While increasing the number of epochs might seem like a potential solution, the additional computational cost often outweighs the gains from Instance-dependent Early Stopping (IES). In other words, the marginal savings achieved by filtering more instances are not sufficient to justify the increased training time introduced by longer training schedules.

2. First-Order or Absolute Loss Signals Are Noisier. While first-order loss differences or even absolute loss values may seem sufficient in low-epoch training regimes, their reliability is undermined by a structural feature of causal language modeling: samples vary greatly in length, and not all tokens contribute equally to the loss. Because loss is typically averaged across tokens in each sample, the variance of this average is inversely proportional to the number of tokens. Consequently, short samples exhibit higher variance in their average loss, making them appear “mastered” more quickly due to statistical fluctuations, even when the model has not fully learned them. This leads to premature exclusion and poor generalization.

3. Non-Uniform Uncertainty Across Samples. The variability in sequence lengths introduces non-uniform uncertainty in the dataset. Shorter samples create volatile loss estimates, while longer samples offer more stable signals. As a result, using unsmoothed loss signals (such as first-order or absolute loss) creates bias in the exclusion mechanism, favoring short and easy samples. To mitigate this, second-order loss differences provide a more robust criterion by smoothing the learning signal and better accounting for natural variations in sample length and token difficulty. However, as previously discussed, this solution cannot be implemented because of the low epoch regime in SFT on LLMs.

Conclusion. While the idea of applying IES to causal language models is theoretically appealing, the structure of SFT training regimes and the nature of sequence-based loss introduce significant limitations. The short training durations restrict the potential benefits of progressive sample exclusion, and the inherent volatility of per-sample loss, especially for shorter sequences, calls for more stable metrics such as second-order loss differences. Future adaptations of IES for LLMs may require new forms of normalization or smoothing tailored specifically to token-level sequence modeling tasks.

Conclusion

The initial motivation of this research project was that forward passes on already mastered instances at every epoch are redundant. Building on Instance-Dependent Early Stopping (IES), we explored how to schedule those re-checks dynamically so that we save computation while preserving accuracy.

My main contribution was to design and benchmark several dynamic forward-pass schedules. The results proved that the most robust approach was the forward pass schedule as a function of the learning rate: as the learning rate decays and the model's updates become smaller, the schedule naturally increases the interval between re-checks, producing stable plateaus and consistent savings across settings. A central difficulty was deciding how fast to increase the interval for mastered instances. If we update too aggressively, we risk under-monitoring, causing a loss in accuracy, but if we are too conservative, we won't see significant savings in computational time. A second challenge was dealing with the sensitivity of derivative-based criteria. Switching from second to first order in the later stages of the learning process could reduce time while keeping a good level of accuracy in theory. In practice, however, our experiments showed that this extension brought no significant improvements to IES, suggesting that it does not meaningfully enhance the method.

The main limitation of our proposed strategies is that dynamic schedules often come with new hyperparameters. To isolate their true effect, other hyperparameters need to be fixed, which demands additional analysis and careful experimental design to fix these newly introduced hyperparameters.

In the second phase, we attempted to extend these ideas to supervised fine-tuning (SFT) of LLMs. This was the most challenging part of the project. The low-epoch regime gives IES little runway to observe stable loss trends, and per-sample loss in sequence modeling is inherently noisier because the number of tokens varies widely between examples. While I did not reach a fully working solution within the constraints of time, resources, and my initial knowledge, the attempt was formative. This experience highlighted the true, iterative nature of research. I truly understood for the first time how patience and rigor work together to come up with meaningful results over the long term in this field.

References

- [1] Suqin Yuan et al. “Instance-dependent Early Stopping”. In: *International Conference on Learning Representations (ICLR)*. Spotlight Presentation. 2025. arXiv: 2502.07547 [cs.LG]. URL: <https://arxiv.org/abs/2502.07547>.
- [2] Alex Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. Tech. rep. University of Toronto, 2009. URL: <https://www.cs.toronto.edu/~kriz/cifar.html>.
- [3] TML Lab. *2025_ICLR_IES*. https://github.com/tmllab/2025_ICLR_IES. GitHub repository, accessed 20 Aug 2025. 2025.
- [4] Kaiming He et al. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*. 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90. URL: <https://doi.org/10.1109/CVPR.2016.90>.
- [5] IBM. *Catastrophic Forgetting*. Accessed: 2025-08-20. 2024. URL: <https://www.ibm.com/think/topics/catastrophic-forgetting>.
- [6] Hiyouga et al. *LLaMA-Factory: Efficient Fine-Tuning Framework for LLMs*. <https://github.com/hiyouga/LLaMA-Factory>. GitHub Repository. 2024.
- [7] Meta AI. *LLaMA 3.2 1B Model Card*. <https://huggingface.co/meta-llama/Llama-3.2-1B>. Accessed on 30 June 2025. 2024.
- [8] Ilias Chalkidis et al. “Biomedical instruction tuning unlocks zero-shot biomedical natural language inference”. In: *Journal of the American Medical Informatics Association* (2024). Accessed on 30 June 2025. DOI: 10.1093/jamia/ocae122. URL: <https://doi.org/10.1093/jamia/ocae122>.

4 Appendix

4.1 Appendix A: Smoothing of second order derivative - Moving Average

The smoothed second-order difference at epoch t is given by:

$$\overline{d_t^{(2)}} = \frac{1}{k} \sum_{i=0}^{k-1} d_{t-i}^{(2)}$$

where:

- $d_t^{(2)}$ is the second-order difference of the loss at epoch t
- k is the size of the moving average window
- $\overline{d_t^{(2)}}$ is the smoothed estimate at epoch t

4.2 Appendix B: Backpropagation with Stochastic Gradient Descent

- 1: **Input:** Learning rate η , number of epochs T , batch size B , loss function $L(\theta; x, y)$
- 2: Initialize parameters θ
- 3: **for** $t = 1$ to T **do**
- 4: **for** each mini-batch $\mathcal{B}$ of size B **sampled uniformly at random** from the training set **do**
- 5: Compute gradient:

$$g \leftarrow \nabla_{\theta} \frac{1}{|\mathcal{B}|} \sum_{(x_i, y_i) \in \mathcal{B}} L(\theta; x_i, y_i)$$
- 6: Update parameters: $\theta \leftarrow \theta - \eta \cdot g$
- 7: **end for**
- 8: **end for**
- 9: **Output:** Final parameters θ_T

- **Momentum:** 0.9
- **Weight decay:** 5×10^{-4}
- **Learning rates and schedulers:**
 - **SGD(F):** Fixed learning rate $\eta = 0.001$.
 - **SGD(L):** Initial learning rate $\eta = 0.1$, with scheduler `LinearLR(start_factor=1, end_factor=0.01, total_iters=150)`.
 - **SGD(E):** Initial learning rate $\eta = 0.1$, with scheduler `ExponentialLR(optimizer, gamma=0.96)`.

4.3 Appendix C: Cross-Entropy Loss

The cross-entropy loss is the standard objective used for classification tasks. It measures the difference between the true distribution y and the predicted probability distribution $\hat{y}$ produced by the model. For a single sample, it is defined as:

$$L(y, \hat{y}) = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

where:

- C is the number of classes,
- y_i is the ground-truth label encoded as a one-hot vector (1 if the sample belongs to class i , 0 otherwise),
- $\hat{y}_i$ is the predicted probability of class i given by the model after applying the softmax function.

This loss is minimized when the predicted probability for the correct class approaches 1.

4.4 Appendix D: Simplified LR based forward pass schedule used for research

```
def k_it_evolve(lr, optimizer_name="SGD_linear"):
    if optimizer_name == "SGD_linear_learning_rate":
        # lr starts at 0.1 and linearly decays to 0.001
        if lr >= 0.05:
            k_it = 1
        elif lr >= 0.02:
            k_it = 2
        elif lr >= 0.01:
            k_it = 3
        elif lr >= 0.005:
            k_it = 4
        else:
            k_it = 5

    elif optimizer_name == "SGD_exponential_learning_rate":
        # exponential decay from 0.1 to 0.001 rapidly
        if lr >= 0.05:
            k_it = 1
        elif lr >= 0.01:
            k_it = 2
        elif lr >= 0.005:
            k_it = 3
        elif lr >= 0.001:
            k_it = 4
        else:
            k_it = 5
    else:
        raise ValueError(f"Unsupported optimizer: {optimizer_name}")

    return k_it
```

Figure 10: Used LR-based forward pass schedule for the research

4.5 Appendix E: Training Arguments for Supervised Fine Tuning

```
train_py_args = [  
    "--model_name_or_path", "Llama-3.2-1B",  
    "--stage", "sft",  
    "--do_train", "true",  
    "--do_eval", "true",  
    "--eval_strategy", "epoch",  
    "--save_strategy", "epoch",  
    "--predict_with_generate", "true",  
    "--metric_for_best_model", "eval_rougeL",  
    "--greater_is_better", "true",  
    "--finetuning_type", "full",  
    "--dataset", "bioinstruct",  
    "--eval_dataset", "bioinstruct",  
    "--load_best_model_at_end", "true",  
    "--template", "llama3",  
    "--cutoff_len", "2048",  
    "--overwrite_cache", "true",  
    "--preprocessing_num_workers", "4",  
    "--dataloader_num_workers", "0",  
    "--output_dir", "saves/llama3-1b/full/sft",  
    "--logging_steps", "10",  
    "--plot_loss", "true",  
    "--overwrite_output_dir", "true",  
    "--save_only_model", "false",  
    "--report_to", "none",  
    "--per_device_train_batch_size", "2",  
    "--per_device_eval_batch_size", "16",  
    "--gradient_accumulation_steps", "2",  
    "--learning_rate", "1e-5",  
    "--num_train_epochs", "5",  
    "--lr_scheduler_type", "cosine",  
    "--warmup_ratio", "0.1",  
    "--bf16", "true",  
    "--deepspeed", deepspeed_config_abs_path  
]
```